

Recipe SpecKit

This repository is an **OC CS SpecKit** monorepo for the **Recipe** application: a Spec-Driven Development (SDD) starter that combines the Recipe product (Vue 3 frontend + Node/Express/Sequelize backend) with SpecKit process docs, Cursor rules, and living reference stubs.

Users create and maintain recipes that have **steps** and **ingredients**, manage a shared **ingredient catalog**, register/sign in, and (when specified) export a recipe PDF.

Important: This kit ships **application code** and the SpecKit **methodology**, but it does **not** ship finished `features/feature-N-*.md` files or product AC tests. Your job is to write feature specs from the running app and the writing guides, then add tests that map to those acceptance criteria.

No application code may be written or changed for a capability unless it maps to an explicit requirement in a feature file (constitution Principle 1).

What is in this repo

Path	Role
frontend/	Vue 3 + Vite + Vuetify Recipe UI (see frontend/frontend_README.md)
backend/	Node + Express + Sequelize API under /recipeapi (see backend/backend_README.md)
features/	SDD home — writing guides, living reference stubs, feature files you add
.cursor/rules/	Stack and agent conventions (how to build)
docs/adr/, docs/nfr/	Architecture decisions and quality attributes

Original Recipe notes (adapted into this monorepo):

- Frontend: create/maintain recipes; local Vite app (this kit uses port **8082**).
- Backend: recipes with steps and ingredients; local MySQL; API on port **3200**; optional Apache ProxyPass to `/recipeapi`.

Methodology & writing guides

Guide	Purpose
features/framework.md	How to write, trace, and ship feature specs
features/writing-feature-requirements.md	Stories, FRs, initial data model, Gherkin AC
features/writing-feature-design.md	Ownership, API, screens, test map, DoD, out of scope
features/reference/writing-living-reference.md	Update api / data-model / behavior when the product changes

(Examples in the writing guides often cite the OC CS SpecKit Todo sample — use them as patterns for the Recipe product.)

Sprints live in your agile tool — they are **not** fields in these specs.

Branch roles: `main` = scaffold-only starter kit · `dev` = integration · `feature/N-*` = feature work (branch from `dev`).

1. Install the project on your machine

1.1 Download and place the project

1. Download the course **zip** of this Speckit Recipe repo.
2. Unzip it wherever you keep class projects (recommended under XAMPP htdocs), for example:

```
/Applications/XAMPP/xamppfiles/htdocs/
```

3. Rename the unzipped folder to **recipe-speckit** so the path looks like:

```
.../htdocs/recipe-speckit
```

1.2 Create a GitHub repository and push this code

1. On GitHub, create a **new empty repository** (no README, no `.gitignore`, no license if GitHub offers them — you already have the project files). Copy the repo HTTPS URL (for example `https://github.com/<your-username>/<your-repo>.git`).
2. Open the **recipe-speckit** folder in **Cursor** (**File** → **Open Folder...** and select **recipe-speckit**). From here on, do the remaining setup **inside Cursor**.
3. Open Source Control: click the **Source Control** icon in the left Activity Bar, or use **View** → **Source Control**.
4. If Cursor shows **Initialize Repository**, click it (this runs `git init` for the folder). If the project is already a git repo, skip to the next step.
5. In Source Control, stage everything: click **+** (Stage All Changes) next to **Changes**, or open the `⌵` menu and choose **Stage All Changes**.
6. In the commit message box at the top of Source Control, type: `Initial commit: Recipe Speckit starter`. Click **Commit** (checkmark). If Cursor asks you to stage first or confirm, accept staging/commit.
7. Publish to GitHub:
 - Click **Publish Branch** (or open the `⌵` menu → **Publish Branch**), or
 - Use the Command Palette (**View** → **Command Palette...**) and run **Git: Add Remote...** / set `origin` to your GitHub HTTPS URL, then **Git: Push**.
 - When prompted, sign in to GitHub if Cursor asks, and choose to publish the current branch as **main** (rename the branch to **main** first if needed: click the branch name in the bottom-left status bar → **Rename...** → **main**).
8. Confirm the GitHub repo shows the monorepo (`frontend/` , `backend/` , `features/` , etc.).
9. Create and publish a `dev` branch:
 - Click the branch name in the **bottom-left status bar** → **Create new branch...** → type `dev` → Create (from **main**).
 - Open Source Control `⌵` menu → **Publish Branch** (or Command Palette → **Git: Push / Publish Branch**) so `dev` exists on GitHub.

For **npm** and other non-git commands later in this guide, use **Cursor's terminal** (**Terminal** → **New Terminal**). For git, prefer the **Source Control** view and status-bar branch menus described above. Use Cursor's editor to create or edit files (for example `.env` and feature specs).

1.3 Backend setup

Work in Cursor. From [backend/backend_README.md](#), adapted for this monorepo:

1. In Cursor's terminal, go to the backend folder and install dependencies by entering:

```
cd backend
npm install
```

(If you are not already at the project root, use `cd /path/to/recipe-speckit/backend` instead.)

2. Create a local MySQL database/schema (example name: `recipe-speckit_db`). Sequelize will create tables on startup.
3. In Cursor's explorer, create `backend/.env` and edit it with correct database values (and secrets):

```
DB_HOST=localhost
DB_USER=root
DB_PW=
DB_NAME=recipe-speckit_db
SECRET_KEY=<base64-encoded-32-byte-key>
PORT=3200
NODE_ENV=development
```

Generate a `SECRET_KEY` with Cursor: open Chat (or Agent) and paste the following prompt (copy the indented lines):

```
Generate a cryptographically secure SECRET_KEY for backend/.env.
In the project terminal, run this command and use its output as the key value:
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
Set SECRET_KEY in backend/.env to that printed value.
Do not commit .env. Leave DB_HOST, DB_USER, DB_PW, DB_NAME, PORT, and NODE_ENV as I already
set them unless the file is missing those keys.
```

Review the change Cursor makes to `backend/.env`, then save the file.

4. In Cursor's terminal (from the `backend` folder), start the API by entering:

```
npm start
```

Server listens on <http://localhost:3200> (routes under `/recipeapi`). Keep this Cursor terminal panel open while the API is running.

5. **(Optional — Apache proxy)** If you want Apache to forward `/recipeapi` to Node (original Recipe setup):

- In XAMPP, edit Apache `httpd.conf`.
- Enable proxy modules (uncomment `LoadModule proxy_http_module` and `LoadModule proxy_http2_module`).
- Add as the last line: `ProxyPass /recipeapi http://localhost:3200/recipeapi`
- Restart Apache.

For local Vite development against Express directly, the frontend can call `http://localhost:3200/recipeapi/` (no proxy required).

1.4 Frontend setup

Still in Cursor: open a **second** terminal panel (`Terminal` → `New Terminal`) so the backend can keep running. From [frontend/frontend_README.md](#), adapted for this monorepo:

1. In the new Cursor terminal, go to the frontend folder and install dependencies by entering:

```
cd frontend
npm install
```

(From project root; or `cd /path/to/recipe-speckit/frontend .`)

2. Ensure **Apache/MySQL** (XAMPP) are available if you use them for the database; the Vite app itself is served by Node.
3. In that Cursor terminal (from the `frontend` folder), run the Vite dev server by entering:

```
npm run dev
```

Keep this terminal panel open while the frontend is running.

4. Open <http://localhost:8082> in a browser (this SpecKit layout uses port **8082**; the original standalone frontend README mentioned 8081).
5. Optional production build — in a Cursor terminal (from the `frontend` folder), enter:

```
npm run build
```

1.5 Smoke-check

- Backend: in a browser, open <http://localhost:3200/> — you should see a welcome message for the recipe backend.
- Frontend: login / published recipes UI loads at <http://localhost:8082>.
- CORS: backend must allow the Vite origin (`http://localhost:8082`).

2. Write feature specification files

Work in Cursor with the `recipe-speckit` folder open. This repo starts with **no** `features/feature-N-*.md` product specs. Explore the running Recipe app (and existing `frontend/` / `backend/` code in Cursor's explorer), then author specs in Cursor's editor before treating a capability as "done" under SDD.

2.1 Make your own feature list (Team)

Before you write any `feature-N-*.md` file, decide how to slice the Recipe app into **features**. Do this in Cursor (notes in a markdown file under `features/` , or draft rows in the catalog in §2.3). Read **Feature vs user story** in [features/writing-feature-requirements.md](#) first.

What is a feature?

A **feature** is a shippable slice of the product: one capability area that can live on its own Git branch (`feature/N-short-name`), merge to `dev` , and leave the app usable. One feature file (`features/feature-N-short-name.md`) owns that whole slice.

A **user story** is one outcome *inside* a feature (for example "sign in" or "add a recipe") — not its own file and not its own branch. Several stories share one feature's screens, data, and release.

How to build your list

1. Run the Recipe app and click through login, recipes, ingredients, edit recipe (steps/ingredients), and any PDF/export controls.
2. In Cursor, list the **capabilities** you see (noun-style names: "User Authentication", "Recipe Management", not "Click Login").

3. For each capability, note a short kebab-case name, whether it depends on another capability, and a few example user stories (not full Gherkin yet).
4. Assign sequential Feature IDs (1 , 2 , 3 , ...). Put later features that need earlier ones under **Depends on**.
5. Check your list against the naming tips in [features/writing-feature-requirements.md](#) (capability, not a single action; product language, not stack jargon).
6. When you are happy with the split, copy the rows into the **Feature catalog** in §2.3 (file names can stay blank until you write each spec). Then proceed to §2.2 for the first feature.

You own this list — there is no required “official” split for the course starter. Use the writing guides and the running app; refine IDs and names if you discover a better slice later (prefer clarifying before you merge a lot of work).

2.2 How to write each feature file (Team Member)

For this course, **you** write the requirements half and the **data model**. You do **not** need to author the other design sections by hand (ownership, API, screens, test map, agent prompt, DoD). Use Cursor for those after your requirements and data model are solid. Full design guidance still lives in [features/writing-feature-design.md](#) if you want to read ahead.

1. Read [features/framework.md](#) for overall section order (so you know what a complete feature file looks like). Choose Feature ID `N` and kebab-case `short-name` (must match the branch and file name).
2. **Create the feature branch** with Cursor menus (from `dev`, not `main`):
 - Click the branch name in the **bottom-left status bar**.
 - Choose **dev** (switch to it). If Cursor offers to pull/sync, sync so you have the latest `dev`.
 - Click the branch name again → **Create new branch...** → type `feature/N-short-name` → Create.
 - Example name: `feature/1-user-authentication-session-management`.
 - Confirm the status bar shows that branch name. Do all of the following work on this branch.
3. In Cursor, create `features/feature-N-short-name.md` (same short name as the branch).
4. **You write the requirements** half using [features/writing-feature-requirements.md](#):
 - Header (`Feature ID` , branch pattern `feature/N-short-name` , status, input, depends on)
 - User stories (`US-N.n`)
 - Functional requirements (`FR-00N`)
 - Assumptions, edge cases, success criteria (`SC-00N`)
 - Key entities
 - Acceptance criteria (Gherkin under each `### US-N.n`)
5. **You write the Data Model Requirements** for this feature (tables, columns, associations this feature needs). Sketch from the running app and existing models under `backend/app/models/`. See the data-model parts of [features/writing-feature-requirements.md](#) and [features/writing-feature-design.md](#).
6. **Cursor writes the rest of the design** (you review and edit). In Chat/Agent, `@` your feature file and paste a prompt like:

```
Complete the design sections for @features/feature-N-short-name.md using @features/writing-
feature-design.md and @features/framework.md.
Keep my Requirements, Gherkin, Key Entities, and Data Model Requirements unless they
contradict the running Recipe app – if something must change, ask me first.
Add or fill: Data Ownership & Isolation, API Requirements, Screen Requirements, Test Coverage
Map, Agent implementation request, Definition of Done, and Out of Scope.
Align API/screens with the existing frontend and backend code where this feature already
exists. Do not invent product behavior that is not in my FRs or Gherkin.
```

- 7. In Cursor, edit this README's **Feature catalog** table (§2.3) and add a row for the new feature.
- 8. Set **Status** to `Ready` only after you have reviewed Cursor's design sections and they match your stories, FRs, Gherkin, and data model.

2.3 Feature catalog

ID	File	Branch	Depends on
—	<i>Add feature-1-...md after you write the first spec</i>	feature/1-...	—

New features: you own requirements + data model; Cursor helps with the remaining design sections. Still follow [features/framework.md](#) for the overall shape of the file.

Stay on `feature/N-short-name` and continue to §3 to add tests. Commit and publish after the tests are in place (§3 step 7).

3. Add tests for the feature you just wrote (Team Member)

Still in Cursor. Do this **for the one feature you finished in §2.2** — not for every feature at once. When you finish Feature N's spec, add and run tests for Feature N only; then repeat §2–§3 for the next feature.

Product AC tests are **not** included in this starter. After that feature's Gherkin and Test Coverage Map exist:

- 1. Open the feature file you just wrote (for example `features/feature-1-...md`). Confirm every `#### Scenario:` appears in its **Test Coverage Map**.
- 2. Follow [features/framework.md](#) **Test traceability** and `.cursor/rules/testing-standards.mdc` for **this feature only**:
 - o File header: Feature N + path to that spec
 - o Outer `describe("Feature N - ...")`
 - o Inner `describe("US-N.n - ...")` matching that feature's AC headings
 - o `it("...")` using the **exact** Gherkin scenario titles from that feature
- 3. Ask Cursor to add the tests. In Chat/Agent, @ **only that feature file** and paste this prompt (copy the indented lines):

```
Add automated tests for @features/feature-N-short-name.md only.
Follow its Test Coverage Map and @.cursor/rules/testing-standards.mdc.
Create or update only the test files listed for this feature (backend/tests and/or
frontend/tests).
Do not add tests for other features. Map every Gherkin scenario in this file to an it() with
the exact scenario title.
Use backend/.env.test for API tests. When done, tell me which files to run.
```

- 4. Review the new/changed test files for that feature. Reject extras that cover a different feature.
- 5. Run **this feature's** tests from Cursor's terminal (use the paths Cursor reported; examples below). Prefer running the specific files over the whole suite.

Backend example — in Cursor's terminal, enter:

```
cd backend
npx jest tests/auth.test.js
```

Frontend example — in Cursor's terminal, enter:

```
cd frontend
npx vitest run tests/Login.test.js
```

Replace those filenames with the ones for **your** feature. If you need the full package suite later, you can still run `npm test` from `backend` or `frontend`.

6. Fix failures until **this feature's** mapped tests pass. Update living reference if this feature changed API/schema/rules.
7. **Commit and publish this feature branch** with Cursor's Source Control UI (status bar should still show `feature/N-short-name`). Include the spec, catalog row, and the new tests together:
 1. Open **View** → **Source Control** (or the Source Control icon in the Activity Bar). Review the changed files (feature markdown, this README's catalog row, and the test files Cursor added for this feature).
 2. Stage only this feature's files: hover each file and click **+**. Typical set: `features/feature-N-short-name.md`, `README.md` (catalog), and the matching `backend/tests/...` and/or `frontend/tests/...` files. Do **not** stage `.env` or unrelated files.
 3. In the commit message box, type a clear message that names the feature (example: `Add Feature 1 user authentication specification and tests`). Click **Commit** (checkmark).
 4. Publish the branch to GitHub: click **Publish Branch**, or open the `⌵` menu → **Publish Branch / Push**. If the branch was already published, use **Sync** or **Push** instead.
 5. Confirm on GitHub that branch `feature/N-short-name` exists and contains your commit. Open a PR into `dev` when your team's process says this feature's spec and tests are ready to merge.
8. Then move on to the next feature (back to §2.2), or stop if you are done for now.

Living reference

Keep snapshots in sync when schema, API, or product rules change — **in the same PR as implementation** (required DoD; see [Merge checklist + Agility sync](#)). Each `feature-N-*.md` should include an **Agent implementation request** so Cursor updates reference during implementation.

File	Purpose
features/reference/README.md	How to maintain reference docs
features/reference/writing-living-reference.md	Student guide — writing/updating living reference
features/reference/data-model.md	Current database tables
features/reference/api.md	Current REST API
features/reference/behavior.md	Current product rules

Related

- Original frontend notes: [frontend/frontend_README.md](#)
- Original backend notes: [backend/backend_README.md](#)
- Cursor rules: `.cursor/rules/`
- ADRs: [docs/adr/](#)
- Quality attributes (NFRs): [docs/nfr/](#)
- Starter kit notes: [docs/STARTER-KIT.md](#)